Old algorithms for new problems

CSE 202, Fall 2026

Russell Impagliazzo

rimpagliazzo@ucsd.edu , 4248 CSE

Dijkstra's ALGORITHM, HIGh LEVEL

procedure dijkstra(G,ℓ,s)
X: set of vertices we know distances to
F: set of vertices we aren't sure of the distance to

Initialize X to be empty, F = V. Initialize dist(s): =0, dist(u) := infinity for other u.
Initialize prev(u) to be nil for all u

Until F is empty:
Find the vertex $u \in F$ with smallest dist(u).
For each edge $(u, v) \in E$ do:
dist(v)= min(dist(v), dist(u)+l((u,v))) (Note: this step is called "relaxing'')
If dist(v) changed, set prev(v)=u
Move u from F to X (Note: this step is called ``ejecting'' u)
Return the arrays dist(u), prev(u)

Useful invariants

- Lemma: after any number of iterations, for every v ∈ F,

  dist(v) = $\min_{u \in X, (u,v) \in E} dist(u) + l((u,v))$ (where, if no such u exists, we view the minimum as infinity). Pred(v) is a u that achieves this minimum.

Proof: After the first iteration, the only possible u is u=s, and the formula is true for N(s), and infinite for other v.

When we add something to X, we update the minimum distances above.

Proving correctness

- Let truedist(v) be the actual minimum path distance from s to v.

- We prove as an invariant that, for all $u \in X$, $truedist(u) = dist(u)$.
- True at start, since truedist(s)=dist(s)=0

Proving correctness: induction step

- Assume at the start of an iteration, $for\ all\ v\ \in X, truedist(v) = dist(v)$. Let u be the vertex we move into X in the iteration, i.e., the element of F with minimum dist value. Let v=pred(u).

  Note that $dist(u) = \min_{\{u' \in F\}} dist(u') = \min_{\{v' \in X, u' \in F, (v',u') \in E\}} (dist(v') + l((v',u')))=$

$$\min_{\{v' \in X, u' \in F, (v',u') \in E\}} (truedist(v') + l((v',u')))$$

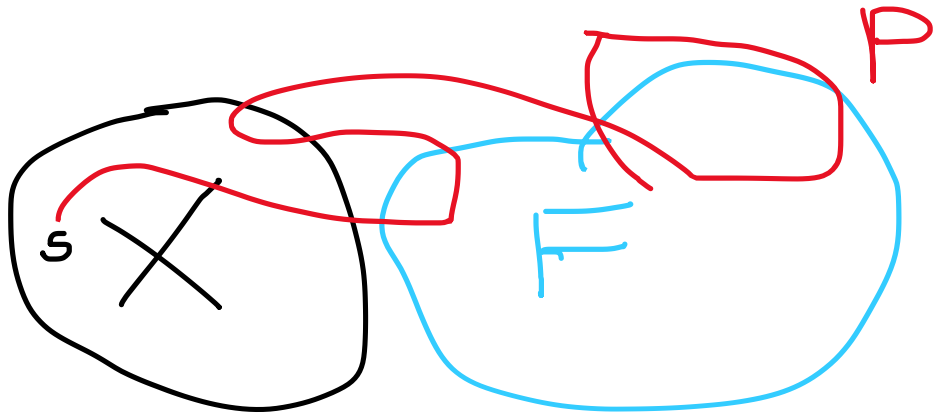
I claim this is the shortest distance from s to anywhere in F.

Proving correctness: induction step

- Let u be the vertex we move into X in the iteration,

Let v=pred(u). $dist(u) = \min_{\{v' \in X, u' \in F, (v',u') \in E\}} (truedist(v') + l((v', u')))$

I claim this is the shortest distance from s to anywhere in F.

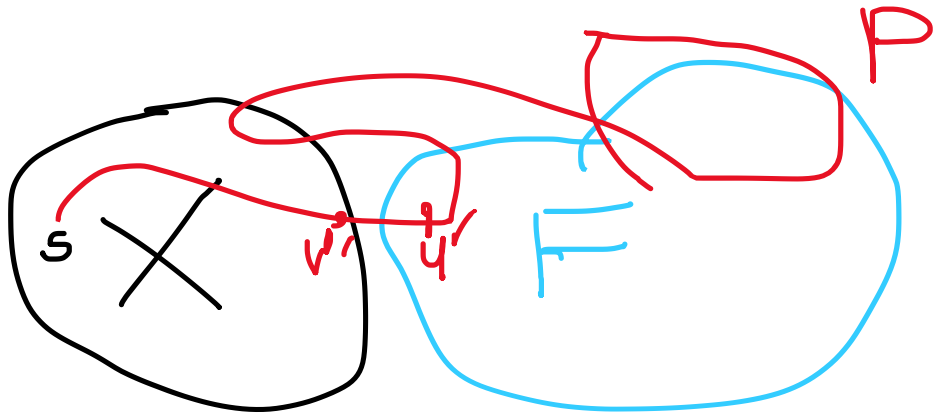


Proving correctness: induction step

- Let u be the vertex we move into X in the iteration,

Let v=pred(u). $dist(u) = \min_{\{v' \in X, u' \in F, (v',u') \in E\}} (truedist(v') + l((v', u')))$

I claim this is the shortest distance from s to anywhere in F.



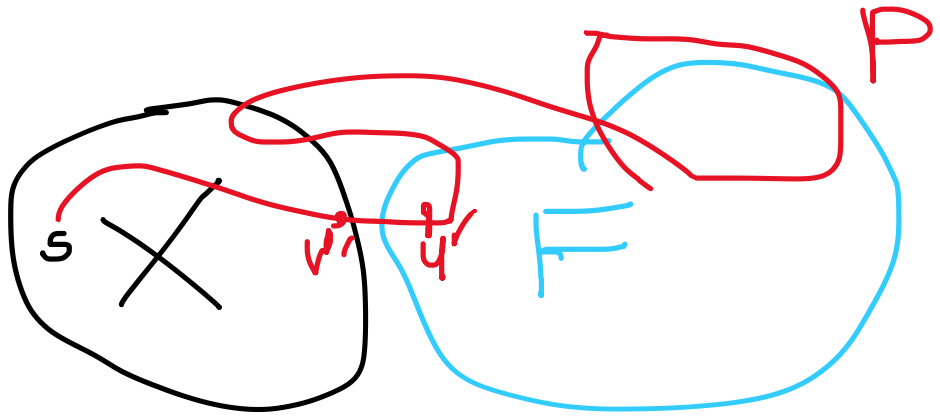
Let P be any path from s to some vertex in F.
Let (v', u') be the first edge in P with u' ∈ F
Let P' be the part of P up to u'. length(P') is at most the length of P (since edge lengths are non-negative). length P' ≥ $truedist(v') + l((v', u')) \geq dist(u)$. *So no path to F can be* shorter than the one we found to u.

Proving correctness: induction step

- Let u be the vertex we move into X in the iteration,

Let v=pred(u). $dist(u) = \min_{\{v' \in X, u' \in F, (v',u') \in E\}} (truedist(v') + l((v', u')))$

I claim this is the shortest distance from s to anywhere in F.



$So\ no\ path\ to\ F\ can\ be$ shorter than the one we found to u.

In particular, no path to u can be shorter than the one we found to u, dist(u)=truedist(u).

Summary: Dijkstra's secret recipe

- Instead of finding the best path to somewhere in particular,
- Dijkstra's algorithm repeatedly finds the best path to ANYWHERE new

- If we have a particular destination in mind, this path might be irrelevant. But when we find the destination we care about, we are guaranteed to have found it by the best possible path.

Priority Queue

A priority queue is a data structure of a set of objects (vertices) along with key values for each object that can be changed (alarm settings.) And it can support the following operations.

- insert(H,u)
- deletemin(H)
- decreasekey(H,u)
- makequeue(S)

Runtime of Dijkstra's

```
procedure dijkstra(G,ℓ,s)
for all u in V
        dist(u) := infinity
        prev(u) := nil
dist(s) := 0
H := makequeue(V)
while H is not empty
        u := deletemin(H)
        for all edges (u,v) in E
                if dist(v) > dist(u) + ℓ(u,v) then
                        dist(v):= dist(u) + ℓ(u,v)
                        prev(v):=u
                        decreasekey(H,v)
```

deletemin × |V|
decreasekey × |E|

LOW-LEVEL DIJKSTRA

- We've reduced the problem of creating a low-level efficient version of Dijkstra's algorithm to that of finding a good implementation of a priority queue
- Total time will be O(n * Time to delete min+ m * time to decrease key)
- Different implementations of priority queue will have different trade-offs between costs of operations. Might not be a single one that is optimal for all kinds of graphs. So flexibility is useful

Array as a priority queue

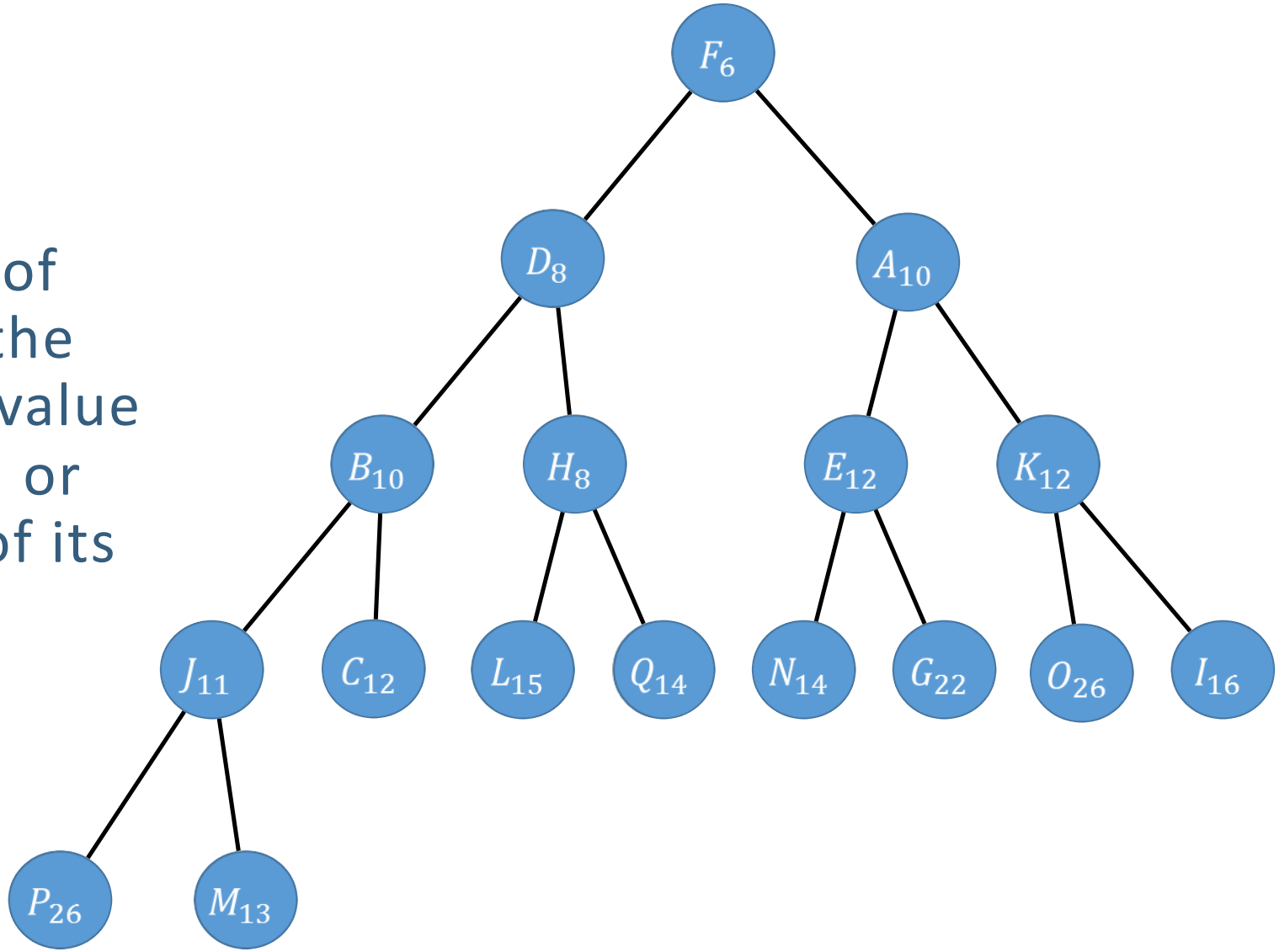
- Array: indexed by vertex, giving key value.

[key(A),key(B),key(C),key(D),key(E),key(F)]

- makequeue:
- deletemin: O(n) time linear search, replace key with X
- decreasekey: O(1): write to array

Binary heap

A complete binary tree of objects (vertices) with the property that each key value of an object is less than or equal to the key value of its children.



1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

$$[F_6, D_8, A_{10}, B_{10}, H_8, E_{12}, K_{12}, J_{11}, C_{12}, L_{15}, Q_{14}, N_{14}, G_{22}, O_{26}, I_{16}, P_{26}, M_{13}]$$

ONE COMPLICATION

- How do we know where v is in the binary heap?

- Keep a supplemental array (an address book) indexed by v, and keep pointers in both directions between this array and binary heap elements.
- This will be our default when we use heaps as priority queues.

Binary heap

- deletemin: $O(\log(|V|))$
- decreasekey: $O(\log(|V|))$

- Dijkstra's takes time:
- deletemin $\times|V|$ + decreasekey $\times|E|$
- $O(\log|V|)\times|V| + O(\log|V|)\times|E| =$
$O\big((|V| + |E|)\log|V|\big)$

Array vs Binary Heap

	• Array $O(\|V\|^2)$	• Binary Heap $O((\|V\| + \|E\|)\log(\|V\|))$
• Sparse graphs: $\|E\| = \Theta(\|V\|)$		
• Dense graphs: $\|E\| = \Theta(\|V\|^2)$		

Array vs Binary Heap

	• Array $O(\|V\|^2)$	• Binary Heap $O((\|V\| + \|E\|)\log(\|V\|))$
• Sparse graphs: $\|E\| = \Theta(\|V\|)$	$O(\|V\|^2)$	$O(\|V\| \log \|V\|)$
• Dense graphs: $\|E\| = \Theta(\|V\|^2)$	$O(\|V\|^2)$	$O(\|V\|^2 \log \|V\|)$

b-ary heap

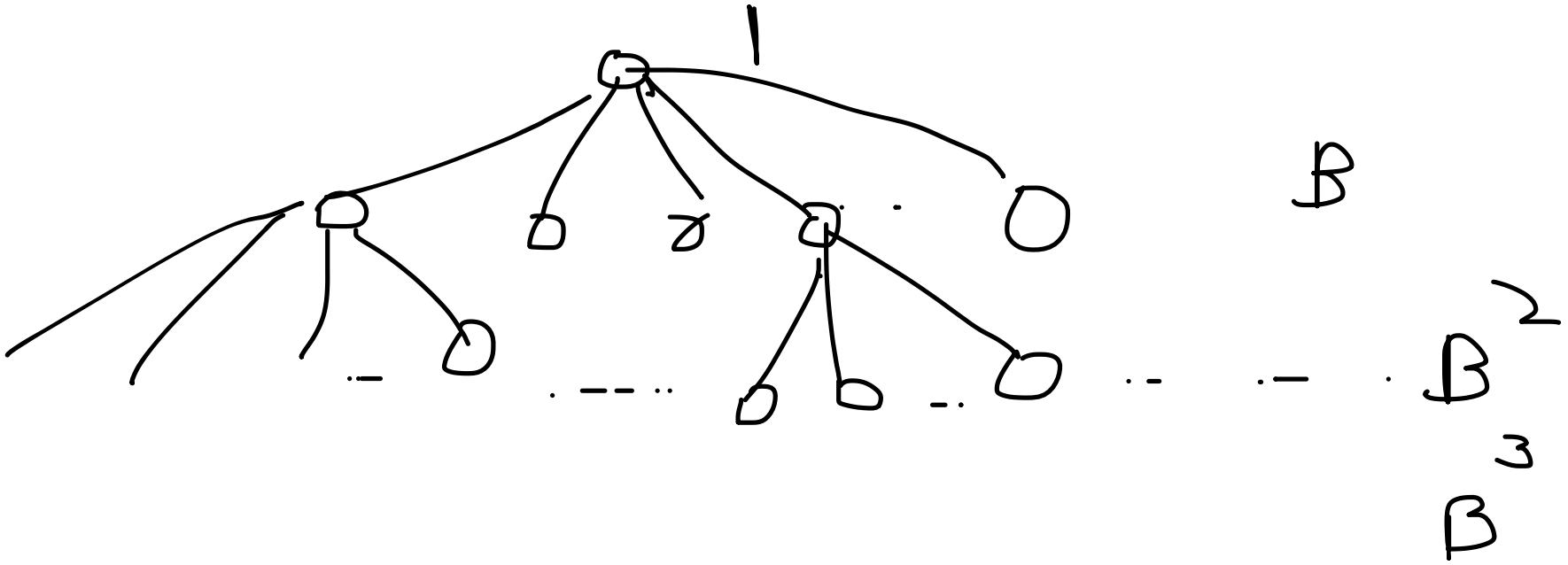
- What if we kept the heap idea, but made it larger fan-out?
- Instead of one child, each vertex has B children.

- When B is really large, almost like an array.

- Depth: $\log_B n = logn/\log B$
- Asymmetry: moving a vertex up the tree, only need to look at parents.
- Moving a vertex down, we need to find the smallest child

b-ary heap

- Depth: $\log_B n = logn / \log B$
- Asymmetry: moving a vertex up the tree, only need to look at parents.
- Moving a vertex down, we need to find the smallest child



b-ary heap

- Depth: $\log_B n = logn/\log B$

- Asymmetry: moving a vertex up the tree, only need to look at parents

So reducing a key value only takes O(depth)= O(log n/log B).

- Moving a vertex down, we need to find the smallest child.

So deleting the minimum element takes time O(B) at each level,
so O(B log n/log B) time.
Total time O((B/log B) n log n + m log n/ log B))= O((Bn+m) (log n/ log B)).

b-ary heap

- Total Dijkstra time: $O((Bn +m) (\log n/ \log B))$.
- Optimization: pick B to set terms equal: $Bn= m$, $B= m/n$.
- Total time $O(m \log n / \log B)= O(m \log n/ \log (m/n))$.

- Note that when m is a power of n between 1 and 2, this is O(m).
- When m = O(n), this is still O(n log n). So improvement for moderate density graphs, but not for sparse graphs.

ADVANTAGES OF FLEXIBILITY

- By working with a higher level version of the algorithm, we can understand what's going on without getting caught up in details
- Flexibility allows us to fit data structures to be efficient for the given circumstance

- Best Dijkstra: PQ using Fibonacci heaps O(nlog n +m) time.

Correctness of Bw-Dijkstra

- Let truebw(v) be the actual max bw from s to v.
- We want to show that at the end of the algorithm bw(v) = truebw(v) for all vertices v.
- In the middle of the algorithm, this is not necessarily true for all vertices. But once a vertex moves to X, its bw is locked in.
- **We will prove that dist(v) = truedist(v) for all v in X.**

- We will prove this by induction on k, the number of elements in X.
- (Note: only one element is put into X at a time.)

Air travel scheduler and max bandwidth

- We'll use graph search to illustrate several topics

1. Formulating new problems
2. Format for problems : instance, solution, constraints , objective
3. Reductions between problems
4. Modification of algorithms

Air travel schedule

- We run an on-line travel agency, like expedia or Travelocity.

- Customers specify their start and destination cities.
- We have a large list of flights available between various cities, including possible transfer points. (Let's think of a flight as on a particular day, not necessarily repeating on subsequent days.)
- We need to find customers a list of flights that meet their needs.
- I'm being purposely vague at a lot of points, here.

For each question:

- Break into groups of about 5 students nearby
- Introduce yourself to the others
- Move if you aren’t near a group
- Discuss the question for about 10 minutes in your group
- End the discussion and sit in your normal seat
- Share group answers in the class in a whole class discussion

Instance (input) .

- What does the input to our scheduler look like?

1) Customer information :

2) Information for each flight:

3) Properties (pre-conditions):

instance

- What does the input to our scheduler look like?

1) Customer information : Starting city S, destination city T, and maybe: time constraints, budget constraints, number of transfers, ????
2) Information for each flight: Flight number f (Think of as 1..n),

Departure city (dc.f), arrival city (ac.f), departure time (dt.f), arrival time (at.f), maybe price (p.f), ?????

Properties (pre-conditions): dt.f < at.f, p.f > 0.

Solution format and constraints on the solution

- What should the output of our algorithm be?

- What properties must it have?

Solution format and constraints

- Format: a sequence of flights, $f_1, \ldots f_k$
- What must be true of a sequence $f_1, \ldots f_k$ to be the sequence of flights the customer can take from S to T?

1)
2)
3)
4)
5)

constraints

- What must be true of a sequence $f_1, .. f_k$ to be the sequence of flights

The customer can take from S to T?

1) $f_1\ must\ depart\ from\ S, f_k\ must\ arrive\ at\ T$
2) Each flight must depart from the city the previous one arrived at
3) Each flight must depart after the previous one arrives (plus time to transfer)

constraints

- What must be true of a sequence $f_1, .. f_k$ to be the sequence of flights

The customer can take from S to T?

1) $f_1\ must\ depart\ from\ S, .DC.f_1 = S;\ f_k\ must\ arrive\ at\ T; AC.f_k = T$
2) Each flight must depart from the city the previous one arrived at $AC.f_{\{i-1\}} = DC.f_i, 2 \leq i \leq k$
3) Each flight must depart after the previous one arrives (plus time to transfer) $at.f_{i-1}(+delay) \leq dt.f_i,\ 2 \leq i \leq k$

objective

- What makes a solution better or worse? This depends on the customer. Let's list some possible objectives as a whole class; then we'll pick two to focus on today.

i. 1
ii. 2
iii. 3
iv. 4
v. 5

Earliest arrival

- Here, the customer wants to get to T as soon as possible, and can leave S immediately. What is the objective function for this goal?

Earliest arrival

- Here, the customer wants to get to T as soon as possible, and can leave S immediately. What is the objective function for this goal?

Objective: Minimize $at.f_k$

Minimum price

- Here, each flight has a price, and the objective is to minimize the total Cost of all flights taken.

Minimum price

- Here, each flight has a price, and the objective is to minimize the total Cost of all flights taken.

Objective: Minimize $p.f_1 + p.f_2 + \cdots p.f_k$

Other objectives are special cases. For example, we can view minimum number of transfers as the case when each flight has price 1.

Flight schedule (Earliest arrival time)

- Instance: Set of flights, $f_1, \ldots f_n,\ with\ each\ f_i\ specifying$:
  DC_i: $departure\ city, dt_i$: $departure\ time, AC_i$: $arrival\ city$, at_i: $arrival\ time$
  with $\mathrm{at_i} > \mathrm{dt_i}$, a start city SC, and a final city FC

Solution format: A sequence of flights, $f_{i_1}, f_{i_2},\ f_{i_k}$

Constraints: $SC = DC_{i_1}, AC_{i_t} = DC_{i_{t+1}}.\ FC = AC_{i_k},\ dt_{i_{t+1}} > at_{i_t}$

Objective: minimize at_{i_k}

How can we use graph search to help?

- Vertices are :

- We put an edge in if :

How can we reduce to graph search?

- Vertices are : flights plus start vertex s

- We put an edge from $f_i \; to \; f_j$ if : $AC_i = DC_j, at_i < dt_j$

and from s to each flight with DC=SC

Return: among the reachable flights, return the smallest arrival time of a flight with destination city FC.

Correctness

- Vertices are: flights plus start vertex s
- We put an edge from $f_i \; to \; f_j$ if : $AC_i = DC_j, at_i < dt_j$ and from s to each flight with DC=SC

Correctness:

The flights for the vertices of any path in this graph is a possible sequence of connecting flights.

Any possible sequence of connecting flights forms a path in this graph.

Proof: The rules for edges are the same as constraints for connecting flights. So a vertex (C,T) is reachable from S if and only if we can arrive at C at time T.

So after we run graph search starting at S, the earliest T so that (FC,T) is in the output is the earliest arrival time.

.

Efficiency (or lack thereof)

- Vertices are : flights plus start vertex s. |V|=n

- We put an edge from $f_i \; to \; f_j$ if : $AC_i = DC_j, at_i < dt_j$

and from s to each flight with DC=SC.

Problem: if there are hub cities,

|E| can reach up to $\Omega(n^2)$, e.g. if ¼ of the flights reach Atlanta in the morning, and ¼ take off from Atlanta in the afternoon.

So even though DFS is ``linear time'', using DFS in this way is quadratic time, $(\Theta(\, n^2)\,)$

How can we reduce to graph search?

- Vertices are : Cities

- We put an edge in if there is a flight, but how do we distinguish arrival times and departure times?

How can we reduce to graph search?

- Vertices are : pairs (City, time), where time is either an arrival time or departure time for that city: at most 2n vertices.

- For each flight f_i we put an edge from (DC_i, dt_i) to $(\mathrm{AC_i}, \mathrm{at_i})$. N edges
- For each city , we make a list of times for that city, and connect them forward in a line (but only to the next time, not all future times). At most 2n edges, one per vertex.
- E.g., (Atl,9 AM) --> (Atl, 10AM) → (Atl , noon) → (Atl, 1 PM) → (Atl , 4 PM)
- Might need to sort events for each city, taking time O(n log n) total.

example

- F1= (SD, LA, 7, 8)
- F2 = (SD, LA, 9, 10)
- F3 = (SD, LV, 10, 11)
- F4= (LA, LV, 6, 7)
- F5 = (LA, LV, 9, 10)

Events at cities: SD: 7, 9, 10

LA: 6, 8, 9

LV: 7, 10, 11,

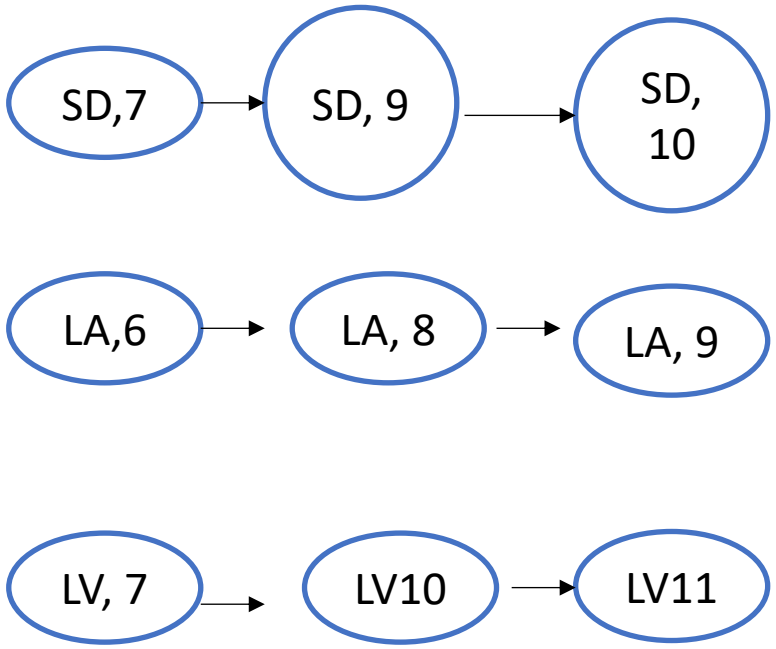
example

- F1= (SD, LA, 7, 8)
- F2 = (SD, LA, 9, 10)
- F3 = (SD, LV, 10, 11)
- F4= (LA, LV, 6, 7)
- F5 = (LA, LV, 9, 10)

Events at cities: SD: 7, 9, 10

LA: 6, 8, 9

LV: 7, 10, 11,



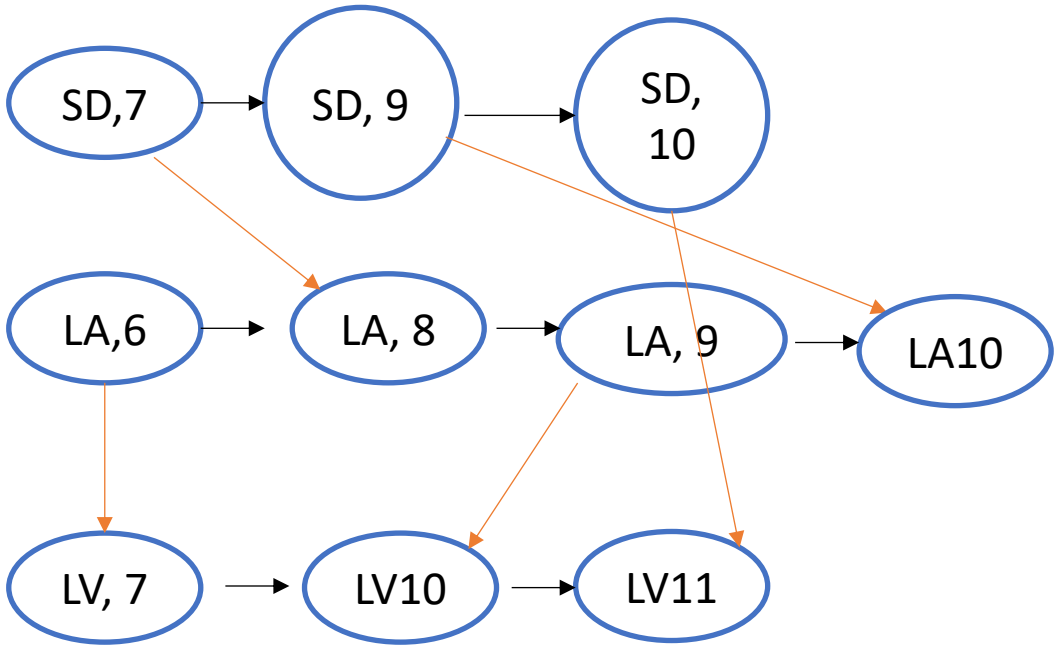
example

- F1= (SD, LA, 7, 8)
- F2 = (SD, LA, 9, 10)
- F3 = (SD, LV, 10, 11)
- F4= (LA, LV, 6, 7)
- F5 = (LA, LV, 9, 10)

Events at cities: SD: 7, 9, 10

LA: 6, 8, 9, 10

LV: 7, 10, 11,



Time

- Time to construct graph: Make list of event (City, time) , two per flight. (2n) Sort list by city name then time. O(n log n). Use this to define edges.

- $|V| \leq 2n$
- $|E|$: waiting edges: $\leq |V| \leq 2n$
- flight edges: exactly n
- So time for Graph search is O(|V|+|E|)=O(n).
- Total time is : O(n log n).

Reduction method

- Reducing from problem NEW to problem OLD
- Problem NEW is: Given x_{NEW} , find a solution y_{NEW} that meets certain constraints$R_{NEW}(x_{NEW}, y_{NEW})$ and possibly maximizes/minimizes an objective function $Objective_{NEW}$ (or say no such y_{NEW} $exists$)
- Problem OLD is: Given x_{OLD} , find a solution y_{OLD} that meets certain constraints$R_{OLD}(x_{OLD}, y_{OLD})$ and possibly maximizes/minimizes an objective function $Objective_{OLD}$ (or say no such y_{OLD} $exists$)

Using reductions to create algorithms:

- Simplest variety: Mapping reduction.
- Given x_{NEW}, describe an algorithm F to compute an instance x_{OLD}
- Use any algorithm to solve OLD to get y_{OLD}
- Describe an algorithm G to convert y_{OLD} to solution y_{NEW}

Correctness proof

- Four parts. Two are more obvious than the other two
- Assume y_{OLD} is a solution to x_{OLD}, $i.e. R_{OLD}(x_{OLD}, y_{OLD})$

- We need to show $y_{NEW} = G(y_{OLD}) is\ a\ solution\ to\ x_{NEW}$: $R_{NEW}(x_{NEW}, y_{NEW})$

- Assume there is no solution to x_{OLD}. We need to show there is no solution for x_{NEW}.

Contrapositive: Give a map H that takes solutions $y_{NEW}\ to\ y_{OLD}$.
Could be inverse to G, but doesn't have to be.

Correctness for optimality

- Show that
  $Objective_{NEW}(\ y_{NEW})\ has\ a\ monotone\ numerical\ relationship$
  to

$Objective_{OLD}(y_{OLD})$. $Need\ same\ relationship\ in\ both\ directions$.

Conclude that best y_{OLD} gives best y_{NEW}

time analysis

- Analyze time for F and G.

Bound relevant size parameters for x_{NEW} from the size parameters for

x_{OLD}.

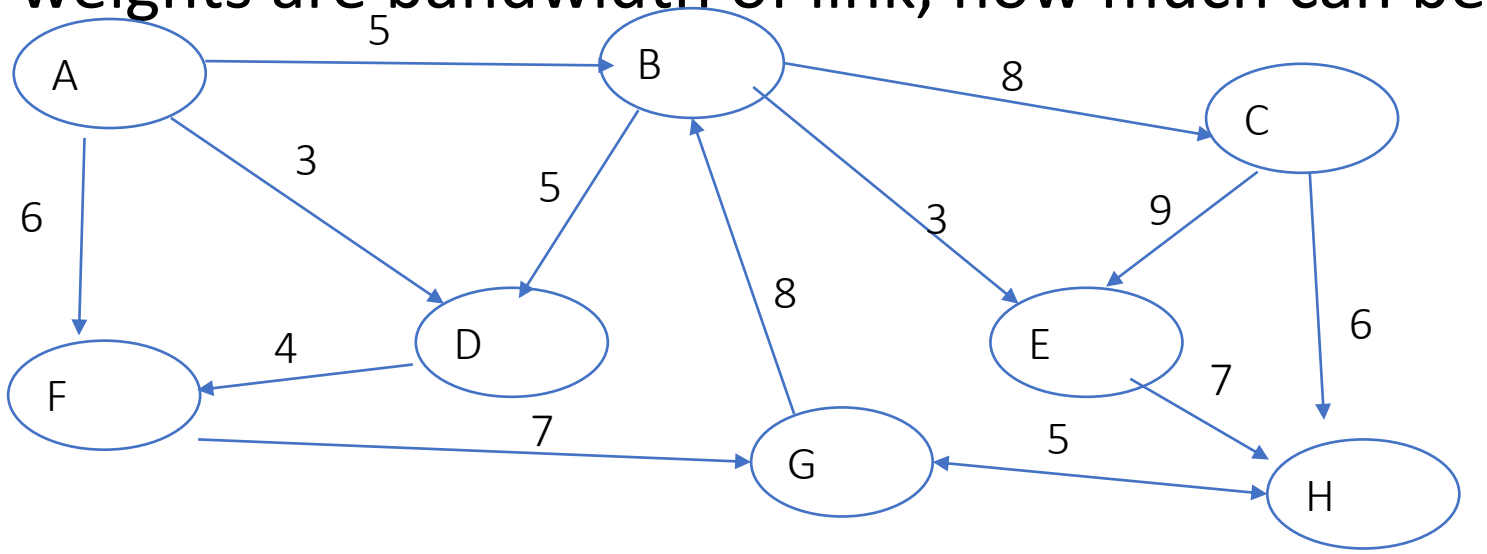
Plug these into a known time analysis for a known algorithm for OLD

Add up these , taking largest (usually the last, but not always)

MAX BANDWIDTH PATH

Graph represents network, with edges representing communication links.

Edge weights are bandwidth of link, how much can be sent



What is the largest bandwidth of a path from A to H?

PROBLEM STATEMENT

- Instance: Directed graph G= (V, E) with positive edge weights, w(e), two vertices s, t ∈ V
- Solution type: a path p from s to t in E.
- Bandwidth of a path:

$$\text{BW}(p) = \min_{e \in p} w(e)$$

- Objective: Over all possible paths p between s and t, find one that maximizes BW(p).

Modifying dikstra's algorithm

- Replace dist with bw
- Reverse initialization: bw(s)=infinity, bw(v)=0 ow
- Replace min priority with max priority
- Replace relaxation with max (bw(v), min(bw(u), b((u,v)))
- I'll let you fill in details

approaches using reachability

- These are approaches that students in past years suggested, modifying or using graph reachability.

- They (mostly) won't be strictly better than the Dijkstra approach,

but will illustrate the merits and dangers of modification vs. reduction,

show that there can be a wide variety of algorithms for the same problem.

Approaches modifying algorithms

- ``Use a stack as in DFS, but instead of keeping just a vertex,

the stack has pairs (V, B), where B is the current bandwidth of

a path from s to V.

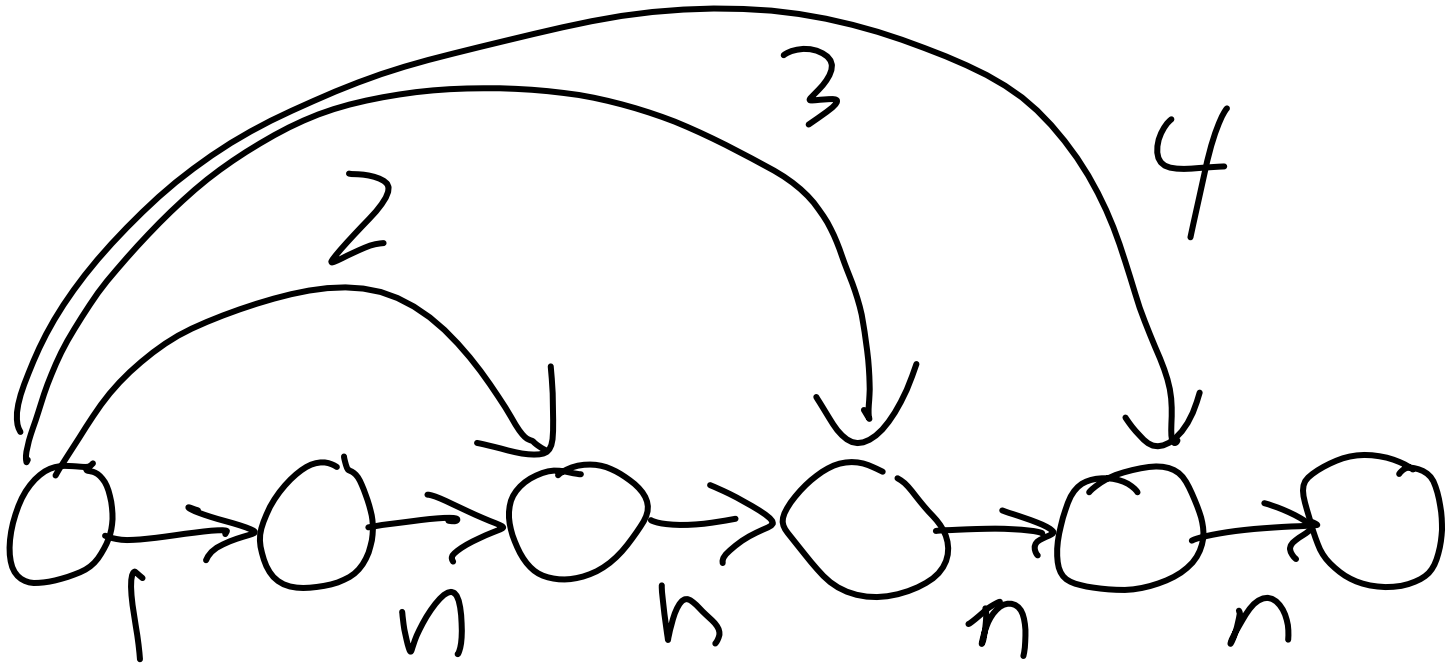
Also keep an array with the best known bandwidth of paths to each u.

When you explore from (V,B), compare the current best bandwidth to

Each neighbor u with the smaller of B and w((V,u)). If the bandwidth to

u improves, update the array and push u with the improved bandwidth''

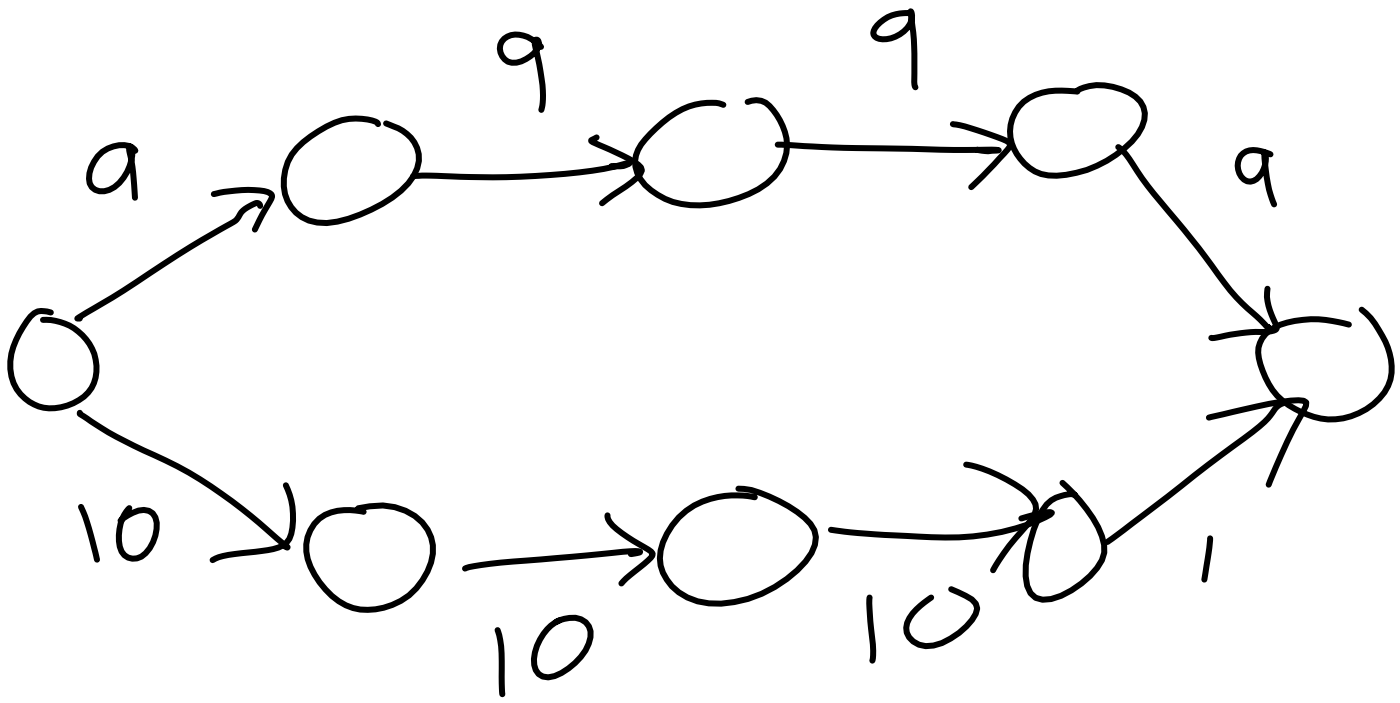
EXAMPLE where this method is quadratic time



Modifying algorithms

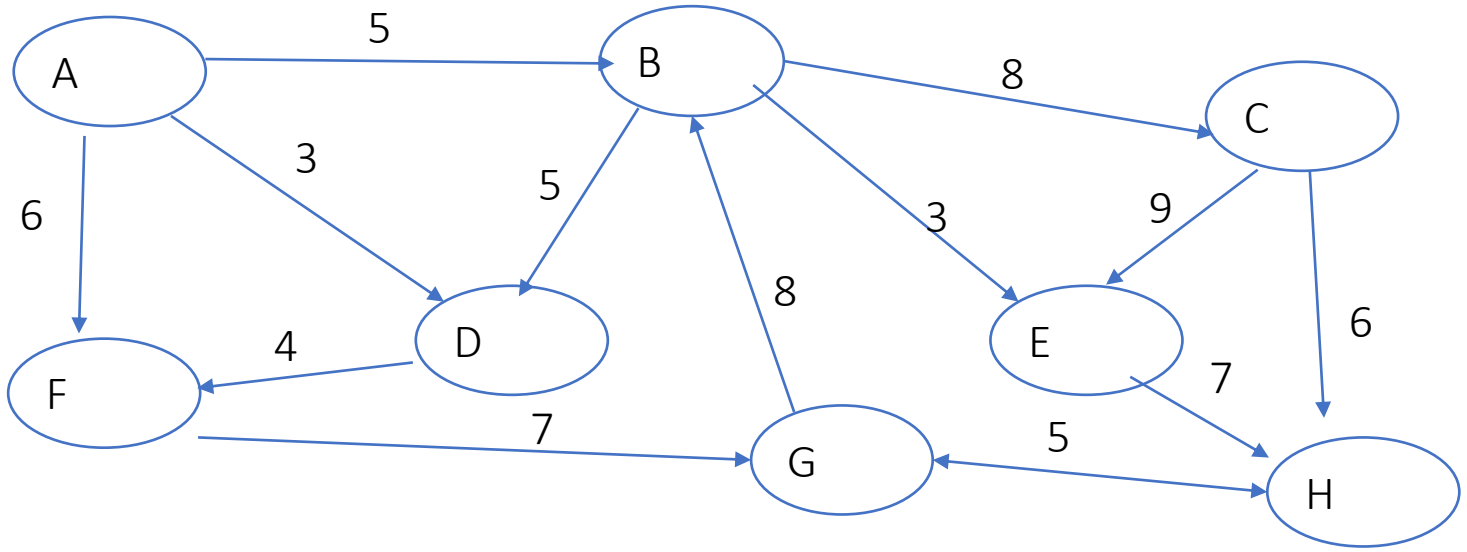
- ``Use DFS, but instead of searching vertices in order of their index, order them by the weight of the edges coming into them''

Problematic instance



Related approach

Student suggested approach: ``Add edges from highest weight to lowest, stopping when there is a path from s to t''



What is the largest bandwidth of a path from A to H?

ReDUCING TO GRAPH SEARCH

Let's phrase this approach as a reduction.

We are using a known algorithm for a related problem to create a new algorithm for a new problem

Here the known problem is : Graph search or Graph reachability

The known algorithms for this problem include Depth-first search and Breadth-first search

In a reduction, we map instances of one problem to instances of another. We can then use any known algorithm for that second problem as a sub-routine to create an algorithm for the first.

A Decision version of the optimization problem

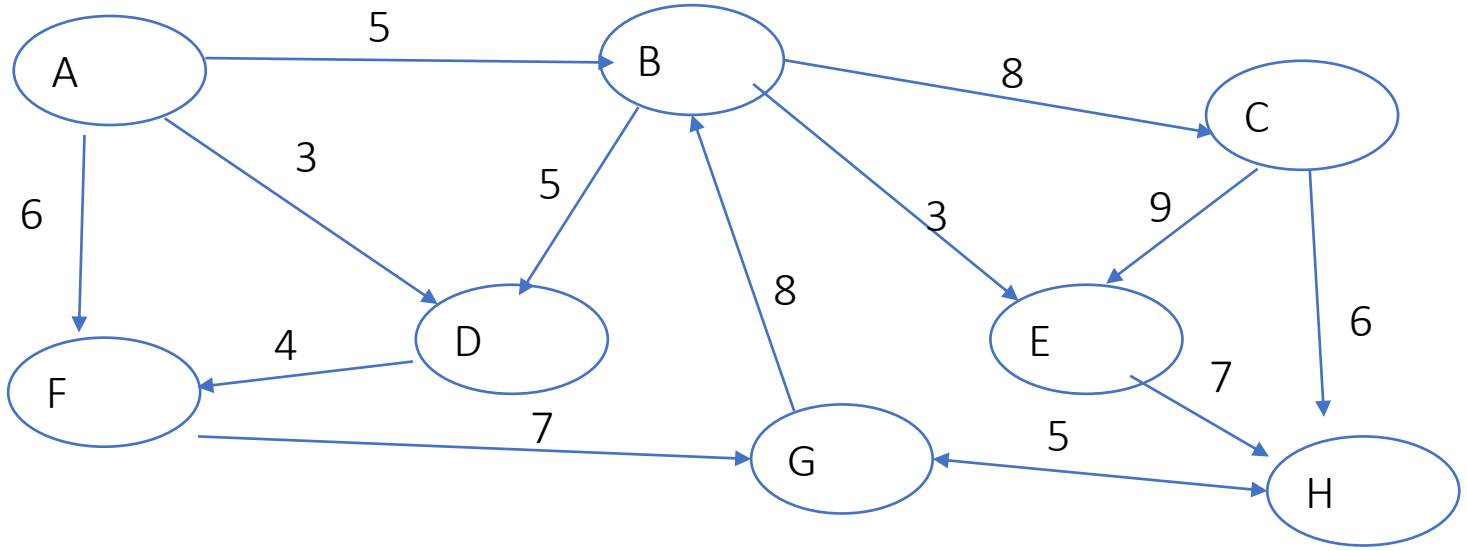
- Reachability is Boolean (yes its reachable, no its not).
- MaxBandwidth is optimization (what is the best bandwidth path)

- To show the connection, let's look at a Decision version of
- Max bandwidth path:

- **Decision Version of MaxBandwidth**
- Given G, s, t, B, is there a path of bandwidth B or better from s to t?

MAX BANDWIDTH PATH

Say B=7, and we want to decide whether there is a bandwidth 7 or better path from A to H. Which edges could we use in such a path? Can we use any such edges?



DECISION TO REACHABILITY

- Let $E_B = \{ e : w(e) \geq B \}$

- Lemma: There is a path from s to t of bandwidth at least B if and only if there is a path from s to t in E_B

DECISION TO REACHABILITY

- Let $E_B = \{ e : w(e) \geq B \}$

- Lemma: There is a path from s to t of bandwidth at least B
  if and only if there is a path from s to t in E_B

Proof: If p is a path of bandwidth BW ≥ B, then every edge in p must have w(e) ≥ B and so is in E_B. Conversely, if there is a path from s to t with every edge in E_B, the minimum weight edge e in that path must be in E_B, so BW(p)= w(e) ≥ B

So to decide the decision problem, we can use reachability:

Construct E_B by testing each edge. Then use reachability on s, t, E_B

Reducing optimization to Decision

- Suggested approach
- ``If we can test whether the best is at least B, we can find the best value by starting at the largest possible one, and reducing it until we get a yes answer. ‘’
- Here, possible bandwidths = weights of edges
- In our example, this is the list: 3, 5, 6, 7, 8, 9
- Is there a path of bandwidth 9? If not,
- Is there a path of bandwidth 8? If not
- Is there a path of bandwidth 7? If not,….

time for this approach

- In each run of Graph search, we are searching on a subset of the original edges, so each run is still O(|V|+|E|) .

We have up to |E| runs.

This makes the time O((|V|+|E|)|E|), much worse than BW-Dijkstra

Can we reduce the number of times we run Graph search?

IDEAS FOR IMPROVEMENT

- Is there a better way we could search for the optimal value?

BINARY SEARCH

- Create sorted array of possible edge weights.
- 3 5 6 7 8 9

- See if there is a path of bandwidth at least the median value
- Is there a path of bandwidth 6? Yes
- If so, look in the upper part of the values, if not, the lower part, always testing the value in the middle

6 7 8 9 Is there a path of bandwidth 8? No

6 7 Is there one of bandwidth 7? No.

Therefore, best is 6

TOTAL TIME FOR BINARY SEARCH VERSION

- How many DFS runs do we need in this version,
- in the worst case? Log |E|
- $|E| \leq |V|^2$.
- log |E| ≤ 2 log |V|, same order.
- Sorting O(|E| log |E|)= O(|E| log |V|)

- What is the total time of the algorithm? O((|V|+|E|) log |V|), same as BW-Dijkstra

MODIFYING GRAPH SEARCH

- This is pretty good, but maybe we can do even better by looking at how graph search algorithms work, rather than just using them as a “black box”

dynamic graph search

- Add edges one at a time (highest weight to lowest)
- Keep X, F and U as before.

F will be empty between adding edges

For each edge (u,v) do:

If u is in X and v is in U do:

Perform a graph search starting at v, using existing edges, and not revisiting vertices already in X (but do add new vertices found to X)

Invariant: After we're done adding edge e and searching, X is set of vertices reachable by bandwidth at least w(e) (if there are no same weight edges, slightly more complex to deal with ties)